FeBEx: In-Network Deduplication for Federated LoRaWAN Backhaul Using P4 Programmable Switches

Savitha Viswanadh Kandala
viswanadh@u.nus.edu

Rajashekar Reddy Chinthalapani
rajashekar.c@u.nus.edu

Abstract

Helium and similar federated LoRaWAN deployments solve the last-mile connectivity problem for IoT devices, but their architecture creates a fundamental inefficiency. When an end device transmits a single uplink packet, multiple geographically distributed hotspots may receive it and independently forward duplicate copies through the backhaul to cloud backends. While this redundancy improves reception reliability and coverage, it also inflates backhaul traffic, increases network cost, and wastes aggregation capacity.

We present FeBEx, a P4-programmable aggregation switch that moves packet deduplication from cloud backends into the network backhaul. FeBEx combines register-based duplicate detection with epoch-driven expiry, longest-prefix-match tenant steering, and packet cloning to preserve proof-of-coverage attribution. We implement FeBEx on a Mininet-BMv2 testbed and evaluate it across deployments with 50–500 devices and 5–50 hotspots. Our results show that FeBEx reduces redundant backhaul traffic by 50–90% while preserving correct packet delivery, maintaining 100% multi-tenant isolation, and enabling accurate payment attribution. We further identify practical operating guidelines, including register sizing of at least $4N$ entries and epoch intervals of 5–10 seconds. These results demonstrate that in-network deduplication is a practical and effective approach for reducing redundant backhaul traffic in federated LoRaWAN infrastructures. Find the code and implementation: `https://github.com/VIS-WA/FeBEx`.

1 Introduction

Helium and other federated LoRaWAN deployments have emerged as an important networking model for large-scale IoT connectivity. In these systems, a large number of geographically distributed hotspots cooperatively receive uplink transmissions from end devices and forward them toward cloud services. This architecture improves reliability and coverage, since multiple hotspots may hear the same transmission even under weak-signal or partially obstructed conditions. However, it also introduces a major inefficiency: the same logical uplink may be forwarded multiple times through the backhaul before duplicate copies are removed at the backend.

This backend-centric deduplication model wastes network resources precisely because duplicate suppression occurs too late. By the time the cloud identifies two packets as duplicates, each copy has already consumed access and aggregation bandwidth. In dense urban deployments, where a single uplink may be received by several hotspots, this behavior can significantly inflate backhaul traffic, increase network cost, and reduce effective aggregation capacity. In Helium-style deployments, the problem is particularly important because the backhaul is shared across independently deployed hotspots and must additionally support accounting mechanisms such as proof-of-coverage attribution.

Design overview. FeBEx addresses this inefficiency by moving deduplication into the network itself. Instead of allowing all copies of an uplink packet to traverse the backhaul and be filtered only at the backend, FeBEx performs duplicate suppression directly at a P4-programmable aggregation switch as shown in Figure 1. This architectural shift allows the first valid copy of a packet to be forwarded immediately, while later duplicates are suppressed before they consume downstream bandwidth. At the same time, the switch must preserve two practical properties required in federated deployments: correct steering of packets to tenant-specific backends, and accurate attribution of the forwarded uplink.

AI Tool Declaration: We used ChatGPT (GPT-5.4 Thinking) and Claude (Opus 4.6 Extended) to generate ideas, improve wording and paragraph structure, refine technical explanations, assist with coding, and help draft and revise parts of this report, including figure captions and section-level writeups. We are responsible for the content, accuracy, and quality of the submitted work.

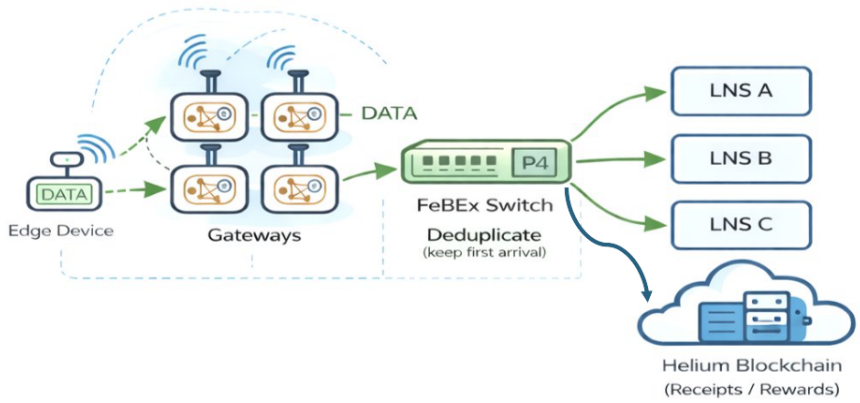

Figure 1: *FeBEx architecture.* Single uplink transmission from an IoT device may be received by multiple Helium hotspots, which forward duplicate copies toward the network. FeBEx, deployed as a P4-programmable aggregation switch, suppresses duplicates in-network, forwards the first valid copy to the tenant backend.

Realizing this functionality in the data plane is not straightforward. The switch must identify duplicates using bounded state, tolerate realistic hash collisions, expire old entries without unbounded growth, and avoid falsely suppressing distinct packets. It must also support multi-tenant forwarding policies while preserving observability of the first forwarded copy for Proof-of-Coverage workflows. FeBEx addresses these requirements through a register-based deduplication pipeline with epoch-driven expiry, longest-prefix-match tenant steering, and packet cloning for attribution.

We implement FeBEx in P4-16 on BMv2 using a P4Runtime control plane and evaluate it on a Mininet-based testbed. Our evaluation studies duplicate suppression correctness, bandwidth savings, state sizing, epoch sensitivity, scalability across different deployment sizes, and multi-tenant isolation. Across the evaluated scenarios, FeBEx reduces redundant backhaul traffic by 50–90% while preserving correct delivery behavior and accurate accounting semantics. We make the following contributions:

- We design and implement FeBEx, a P4-based in-network deduplication mechanism for federated LoRaWAN backhaul aggregation.
- We show that FeBEx suppresses redundant uplink copies before egress, substantially reducing backhaul traffic while preserving correct packet delivery.
- We characterize the impact of register size, epoch interval, and network scale, and derive practical guidelines for provisioning deduplication state.
- We demonstrate that FeBEx preserves multi-tenant traffic isolation and supports attribution alongside duplicate suppression.

2 Background and Related Work

LoRaWAN for IoT Connectivity. LoRaWAN has emerged as a widely used low-power wide-area networking technology for Internet-of-Things (IoT) deployments that require long range, low energy consumption, and low infrastructure cost [1, 2]. In a typical LoRaWAN deployment, battery-powered end devices transmit uplink packets that may be received by multiple gateways within radio range. These gateways forward the packets over IP backhaul to a network server, which performs tasks such as packet deduplication, device management, and delivery to application backends [1, 3, 4]. This architecture improves reliability because successful reception does not depend on any single gateway. However, it also introduces redundancy by design: a single uplink may be forwarded multiple times through the backhaul before duplicate copies are removed at the backend [1, 3]. Prior studies on LoRa and LoRaWAN scalability further show that dense reception and shared infrastructure can create significant capacity pressure as deployments grow [5, 6].

Why Federated IoT Networks Matter. As IoT deployments scale across large geographic regions, a federated network model becomes increasingly attractive. Instead of relying on a single centrally deployed operator infrastructure, federated IoT networks allow many independently deployed gateways

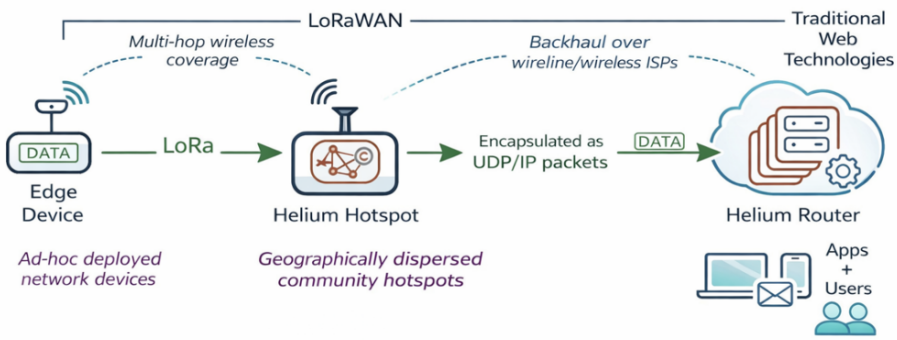

Figure 2: *Helium-style federated LoRaWAN architecture.* Battery-powered end devices transmit uplinks that are received by geographically distributed community hotspots. These hotspots encapsulate LoRa packets into IP traffic and forward them through backhaul links toward shared cloud services and application backends.

or access points to contribute connectivity into a shared service fabric. This model improves scalability, coverage, and deployment incentives, especially in environments where infrastructure density varies across neighborhoods or cities. At the same time, federation shifts some challenges from the wireless edge into the network core. Since independently deployed gateways may all forward traffic toward shared backend services, duplicate receptions are no longer merely a backend software issue; they also consume shared access and aggregation bandwidth. As a result, efficient backhaul handling becomes an important systems problem in federated IoT infrastructure [2, 7, 8, 5, 6].

Helium as a Federated LoRaWAN Deployment. Helium is a prominent example of a federated LoRaWAN-style network, in which connectivity is provided not by a single centrally deployed operator infrastructure, but by many independently operated community hotspots [7, 9, 8]. As shown in Fig. 2, battery-powered end devices transmit LoRa uplinks that may be received by one or more nearby hotspots. Each hotspot acts as a gateway: it demodulates the wireless packet, encapsulates it into standard IP traffic, and forwards it over the backhaul towards cloud [3, 4]. This federated architecture is attractive because it improves coverage and scalability while reducing the need for a single entity to deploy and maintain the entire access infrastructure.

However, the same architecture also creates a major inefficiency. Since LoRaWAN uplinks are broadcast in nature, the same device transmission may be heard by multiple hotspots within range [1, 2, 5, 6]. In current deployments, each hotspot independently forwards its received copy of the packet toward the backend, even though these copies correspond to the same logical uplink [3, 8]. As illustrated in Fig. 3, the backend eventually performs deduplication and preserves correctness at the application layer, but by that point the redundant copies have already consumed backhaul bandwidth. This wastes aggregation capacity and becomes increasingly costly in dense deployments where multiple hotspots frequently hear the same transmission.

Beyond simple delivery, Helium also imposes an additional requirement: attribution. Incentive and accounting mechanisms such as proof-of-coverage and data-transfer rewards depend on preserving information about hotspot participation in the network [7, 10, 11, 8]. Therefore, an in-network deduplication mechanism for Helium must do more than suppress redundant traffic; it must also preserve the attribution semantics required for rewards, auditing, and tenant-specific backend processing.

Deduplication and In-Network Processing. Packet deduplication is commonly performed at the backend in LoRaWAN-style systems, where multiple gateway copies of the same uplink are forwarded to a centralized service and filtered in software. This approach is simple and flexible, but it removes duplicates only after they have already consumed backhaul bandwidth and processing resources. More broadly, intermediate duplicate suppression and in-network aggregation have been explored in wireless sensor networks and edge systems to reduce communication overhead and improve efficiency [12]. Closely related to our setting, prior work has also explored reducing duplicate LoRaWAN gateway forwarding through coding-aware forwarding strategies [13]. However, many such approaches operate at different layers, assume more homogeneous network stacks, or do not address the constraints of shared federated IoT backhaul. In contrast, FeBEx moves duplicate suppression earlier into the network and targets Helium-style LoRaWAN deployments, where duplicate filtering must coexist with tenant-aware forwarding and hotspot attribution [12, 13].

P4 and Programmable Data Planes. P4 enables custom packet-processing logic to be implemented

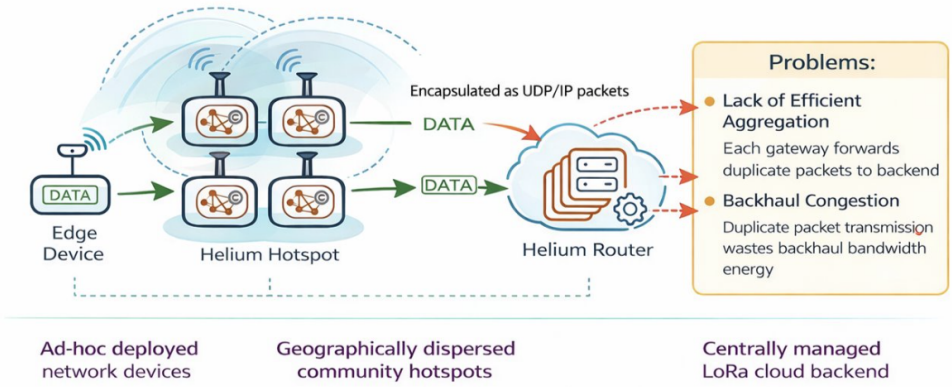

Figure 3: *Redundant uplink forwarding.* A single device transmission may be received by multiple Helium hotspots, which independently forward duplicate copies toward the backend. While backend deduplication preserves correctness, redundant copies still consume backhaul bandwidth before being discarded.

directly in programmable switch data planes [14]. Prior work has used P4 to support stateful network functions such as in-band telemetry [15], load balancing [16], and in-network aggregation [17]. More recent work has further shown that programmable switches can support richer in-network computation primitives beyond conventional forwarding [18]. Together, these systems demonstrate that programmable switches can maintain bounded state, perform packet cloning, and execute application-specific forwarding decisions at line rate. FeBEx builds on the same general capabilities, but applies them to a different problem: duplicate suppression in federated IoT backhaul. In particular, it combines register-based state, match-action forwarding, and cloning to jointly support duplicate filtering, tenant steering, and proof-of-coverage-aware attribution.

Hash-Based Filtering. Compact duplicate detection is often implemented using hash tables or Bloom-filter-like structures [19, 20]. Storing full packet contents or performing exact-match lookup is typically too expensive for switch memory, making bounded-state hashing a more suitable approach in programmable data planes. FeBEx adopts a two-level hash design: one hash selects the register index, while a second hash serves as a verification key to reduce false matches caused by collisions [19]. Compared to probabilistic filters such as Bloom filters [20], this design is simpler to express in P4 and aligns naturally with deterministic register lookups. The tradeoff is that correctness depends on suitable provisioning of register capacity and expiry intervals, which FeBEx explicitly evaluates.

IoT Backhaul and Edge Network Services. Prior work on programmable switches has increasingly explored stateful edge services that move functionality from servers or middleboxes into the forwarding path. Examples include telemetry [15], switch-assisted coordination [21], load-aware forwarding [16], and more general in-network computation [18]. At the same time, LoRaWAN and Helium-style infrastructures face practical backhaul challenges including redundant forwarding, bandwidth cost, and the need for accurate attribution in incentivized deployments [8]. FeBEx sits at the intersection of these directions: it treats duplicate suppression not as a backend-only software task, but as an in-network edge service implemented in the data plane.

Positioning of FeBEx. FeBEx is distinguished by the combination of four properties. First, it is motivated by Helium-style federated LoRaWAN deployments rather than generic packet filtering [8]. Second, it embeds duplicate suppression directly in a programmable forwarding pipeline instead of relying on separate middleboxes or cloud-side processing. Third, it operates under bounded switch state using a compact register-based design with epoch-based expiry and hash-based verification [19, 20]. Finally, it preserves practical deployment requirements beyond correctness alone, including multi-tenant traffic steering and attribution-aware packet cloning. Together, these aspects position FeBEx at the intersection of backhaul optimization, programmable data-plane systems, and compact in-network duplicate filtering.

3 Design

This section describes the FeBEx architecture and the key design decisions that enable in-network duplicate suppression for federated LoRaWAN backhaul. Figure 4 illustrates the overall system and the sequence of processing stages applied to each uplink packet inside the programmable switch. At a high

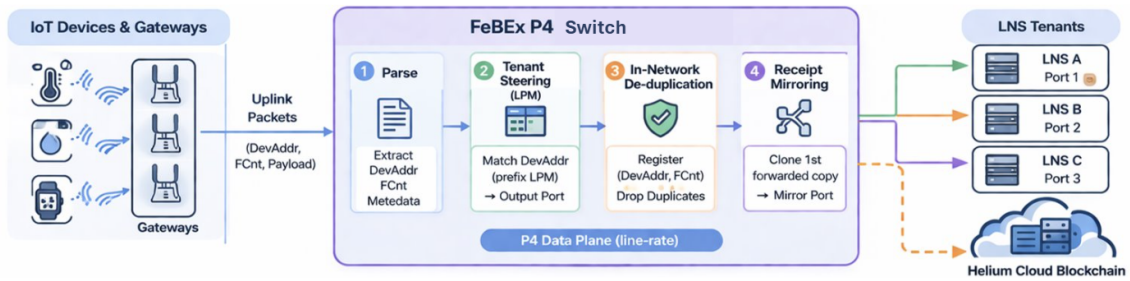

Figure 4: *FeBEx system architecture and packet-processing pipeline.* Uplink packets from IoT gateways arrive at the FeBEx P4 switch, which extracts the relevant metadata, steers packets to the appropriate LNS tenant using DevAddr-based forwarding, suppresses redundant copies using register-based in-network deduplication, and mirrors the first forwarded copy to the Helium cloud path for receipt and attribution handling.

level, FeBEx is deployed as a programmable aggregation switch between multiple gateways and shared backend services. Rather than allowing every received copy of an uplink packet to traverse the backhaul and be discarded only at the backend, FeBEx suppresses redundant copies directly in the forwarding path while preserving tenant-specific routing and attribution semantics.

System Overview. FeBEx consists of two main components: a programmable data plane and a lightweight control plane. The data plane is implemented in P4 on BMv2 and is responsible for per-packet forwarding decisions. The control plane is implemented in Python using P4Runtime and Finsy, and is responsible for installing forwarding rules, configuring clone sessions, and rotating epochs for deduplication state expiry. Within the FeBEx pipeline, each packet is first parsed to extract the metadata fields, then matched to the appropriate tenant backend, checked against deduplication state, and finally cloned if it is the first valid copy of a logical uplink. In this way, FeBEx performs three tasks in a single forwarding pipeline: *(i)* tenant-aware forwarding to the appropriate LoRa Network Server (LNS), *(ii)* in-network duplicate suppression using compact register-based state, and *(iii)* cloning of selected packets to a cloud endpoint for attribution.

Deployment Topology. FeBEx assumes a star-shaped aggregation topology in which multiple gateways connect to a single programmable switch, which then forwards traffic to one of several tenant backends and, when required, to a cloud endpoint. In our Mininet-based implementation, gateway hosts generate packets and inject them into the switch, the BMv2 switch executes the FeBEx pipeline, tenant LNS hosts receive forwarded packets, and a separate cloud host receives cloned receipts. Concretely, if there are K gateways and M tenant backends, gateway hosts connect to ports $1 \ldots K$, tenant hosts connect to ports $K+1 \ldots K+M$, and the cloud host connects to port $K+M+1$. This topology captures the central aggregation role of FeBEx while remaining simple enough for controlled experimentation.

Packet Abstraction. Rather than parsing the full LoRaWAN stack inside the switch, our prototype uses a compact custom metadata header that carries only the fields needed for deduplication, forwarding, and attribution, as shown in Fig. 5. Each test packet consists of Ethernet, IPv4, UDP, a 12-byte FeBEx metadata header, and an arbitrary payload:

```
Ethernet | IPv4 | UDP (port 5555) |  FeBEx Meta  | Payload

                                   |-- 12 bytes --|
```

The metadata header contains four fields of interest: `dev_addr`, `fcnt`, `gw_id`, and two reserved bytes. The pair (`dev_addr`, `fcnt`) identifies a logical uplink, while `gw_id` records which gateway produced the current copy. This abstraction lets us focus on the core network problem—duplicate suppression in shared backhaul—without tying the design to a specific physical-layer packet representation.

Deduplication Logic. The central design problem in FeBEx is deciding whether an arriving packet is the first observed copy of a logical uplink or a redundant duplicate that should be suppressed. To do this within bounded switch memory, FeBEx uses a two-level hash design. First, an *index hash* maps the packet identifier (`dev_addr`, `fcnt`) to a register slot. Second, a *verification hash* is stored in that slot to reduce the chance of treating two different packets as the same entry. Each slot additionally stores an epoch value, allowing stale entries to expire without explicit deletion.

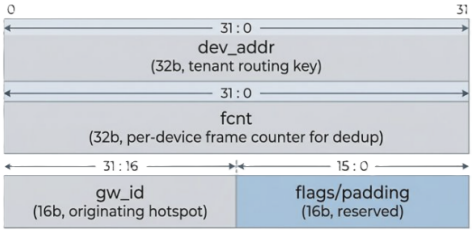

Figure 5: *FeBEx packet metadata format.* Each packet carries a compact 12-byte FeBEx header containing the device address, frame counter, and gateway identifier. The tuple (`dev_addr`, `fcnt`) identifies a logical uplink for duplicate detection, while `gw_id` preserves attribution information for the gateway that forwarded the packet.

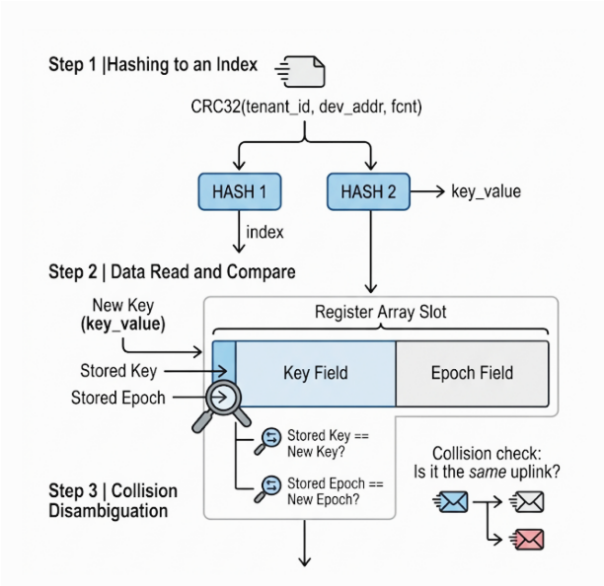

Figure 6: *FeBEx deduplication workflow.* For each incoming packet, the switch computes an index hash and verification key from (`dev_addr`, `fcnt`), reads the corresponding register entry, and compares the stored key and epoch against the current values. A matching key within the active epoch is suppressed as a duplicate; otherwise, the entry is updated and the packet is forwarded as the first valid copy.

Operationally, for each arriving packet, the switch computes an index and verification key from (`dev_addr`, `fcnt`), reads the stored tuple at the selected register slot, and compares the stored key and epoch against the current values, as illustrated in Fig. 6. If both match, the packet is treated as a duplicate and suppressed. Otherwise, the switch updates the slot with the new key and current epoch, and the packet is forwarded as the first valid copy. This design keeps the data-plane state compact while enabling duplicate detection at line rate.

Epoch-Based Expiry. A deduplication entry should not persist indefinitely, since old packets must not suppress future packets that happen to reuse the same identifier space after sufficient time has passed. FeBEx addresses this using epochs. The control plane periodically increments a global epoch counter, and each register entry stores the epoch in which it was last written. On lookup, if the stored epoch differs from the current epoch, the entry is treated as stale and can be safely overwritten. This avoids the need for explicit per-entry deletion and provides a simple mechanism for bounding the lifetime of deduplication state. In practice, epoch intervals in the range of a few seconds provide a good tradeoff between state freshness and duplicate suppression.

Tenant-Aware Forwarding. Because FeBEx is intended for federated and potentially multi-tenant deployments, duplicate suppression alone is insufficient; packets must also be routed to the correct backend. FeBEx performs tenant steering using Longest Prefix Match (LPM) over `dev_addr`. The control plane assigns each tenant a contiguous address prefix and installs the corresponding forwarding entries into the switch. On ingress, the switch matches the packet's `dev_addr` against these prefixes, selects the egress port for the appropriate LNS backend, and rewrites the destination Ethernet address before forwarding. This design is simple, scalable, and avoids maintaining per-device forwarding state in the switch.

Attribution and Receipt Cloning. A key requirement in Helium-style deployments is that duplicate suppression must not destroy attribution information. Although only one packet copy should continue toward the backend, the system must still preserve information about which gateway legitimately delivered the first valid copy. FeBEx supports this through ingress-to-egress cloning. When a packet is identified as the first valid copy of a logical uplink, the switch not only forwards it to the selected tenant backend but also clones it to a cloud endpoint. The cloned packet retains the gateway identifier, enabling downstream Proof-of-Coverage-related accounting and auditing. In our implementation, this is realized using a P4 clone action with a controller-configured clone session.

Control Plane Responsibilities. The FeBEx control plane is intentionally lightweight. Its role is not to process packets directly, but to provide the data plane with the configuration needed for correct operation. Specifically, the controller installs LPM entries for tenant steering, configures the clone session used for receipt mirroring, initializes and updates deduplication-related state, and periodically rotates the epoch value used for entry expiry. This separation keeps the per-packet logic inside the switch fast and simple, while allowing reconfiguration and policy updates at runtime.

Design for Controlled Comparison. To measure the effect of in-network deduplication, FeBEx also supports a routing-only baseline mode in which duplicate suppression is disabled and all packets are forwarded. This mode is controlled through a small register-based toggle in the P4 program. When disabled, the switch still performs tenant-aware routing and cloning, but no packets are suppressed. This allows direct A/B comparison between conventional forwarding and FeBEx-style in-network deduplication under the same topology and traffic conditions.

4 Implementation

This section summarizes the FeBEx implementation at a high level, focusing on how the design is realized in the prototype rather than on code-level details. FeBEx is implemented using three main components. First, the *P4 data plane*, running on BMv2, parses the FeBEx metadata header and performs tenant steering, duplicate suppression, and receipt cloning. Second, a *P4Runtime control plane*, implemented in Python using Finsy, installs tenant forwarding entries, configures clone sessions, and rotates the epoch value used for state expiry. Third, an experiment harness, built using Mininet and Scapy, generates traffic, captures outputs, and computes the metrics used in evaluation.

Packet and Traffic Path. Each gateway sends UDP packets carrying FeBEx metadata of the form (`dev_addr`, `fcnt`, `gw_id`). On arrival at the switch, the packet is parsed, mapped to a tenant backend using `dev_addr`, checked against deduplication state, and either forwarded or suppressed. For each unique logical uplink, FeBEx forwards exactly one copy to the correct tenant backend and mirrors that first-forwarded copy to the cloud path for attribution. Subsequent duplicates of the same logical uplink are dropped in the switch.

Switch Pipeline Flow. The switch ingress pipeline follows a simple sequence of operations, as illustrated in Fig. 7. The parser first extracts Ethernet, IPv4, UDP, and FeBEx metadata fields. The ingress pipeline then performs tenant steering using an LPM table over `dev_addr`, producing the tenant-specific egress decision. Next, the switch executes the deduplication check by computing a hash index and verification key, reading the corresponding register state, and comparing the stored values against the current packet and epoch. If the packet is identified as a duplicate, it is dropped. Otherwise, the corresponding state is updated, the packet is forwarded to the tenant backend, and a clone is generated for receipt handling.

Controller Operation. The controller is responsible for configuring the switch but does not participate in per-packet processing. At startup, it connects to the switch over P4Runtime, loads the compiled pipeline, installs tenant-specific LPM forwarding entries, initializes deduplication-related state, and configures the clone session used for receipt mirroring. During runtime, it periodically updates the global epoch register according to the chosen expiry interval. This design keeps the switch fast and self-contained while allowing runtime control over policy and state lifetime.

Deduplication State Layout. FeBEx uses a compact register-based state design. The register array `dedup_keys[index]` stores a 32-bit verification key for each occupied slot, while `dedup_epochs[index]` stores the corresponding 16-bit epoch value. A separate singleton register, `current_epoch[0]`, holds

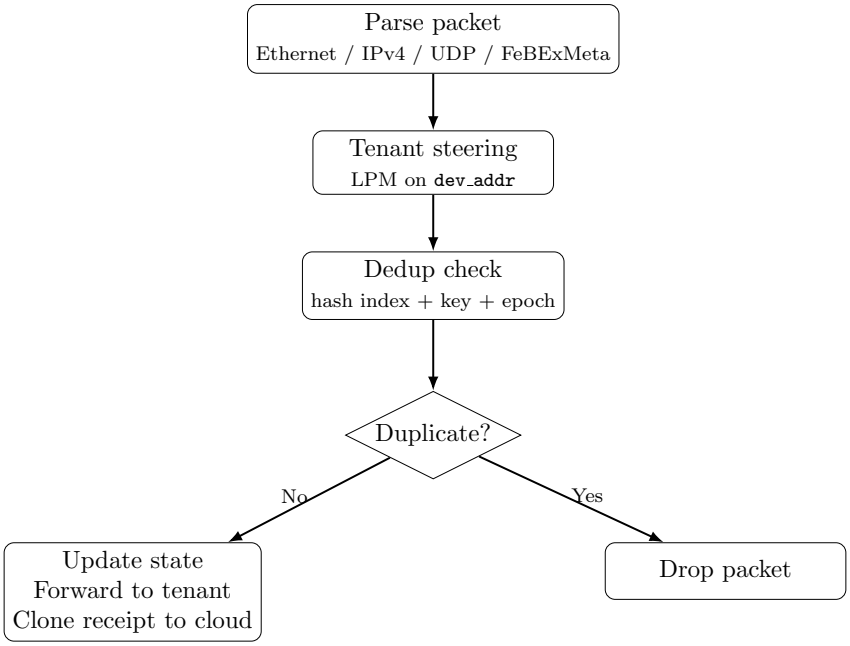

Figure 7: *FeBEx switch ingress pipeline.* Each packet is parsed, mapped to the appropriate tenant backend, checked against deduplication state, and either forwarded with receipt cloning or dropped as a duplicate.

Boundary-Leakage Mitigation Across Variants

Timeline: ... $E_t \mid E_{t+1}$ ... (epoch boundary at |)

V1: check E_{t+1} only
duplicate that first arrived in E_t and repeats in E_{t+1} may leak

V2 (sliding-window): check E_t *or* E_{t+1}
cross-boundary duplicates are still recognized and suppressed

V3 (dual-register Bloom): collision-focused design with stronger FP resistance
improves collision behavior, but does not eliminate boundary leakage by itself

Figure 8: *Conceptual comparison of FeBEx switch variants.* V1 uses a single-epoch check, V2 extends duplicate detection across an epoch boundary, and V3 improves collision behavior through a dual-register Bloom-style design.

the epoch value currently enforced by the controller. Another singleton register, `dedup_enabled[0]`, acts as a runtime toggle for enabling or disabling duplicate suppression during A/B comparisons. This compact layout avoids expensive per-device state while supporting simple and efficient lookup-and-update operations in the data plane.

Switch Variants for Boundary-Leakage Mitigation. To study the effect of epoch-boundary behavior and collision tradeoffs, we implement and compare three switch variants. *V1* is the baseline design, which treats a packet as a duplicate only if the stored epoch exactly matches the current epoch; this makes it vulnerable to boundary leakage when repeated packets straddle an epoch transition. *V2* extends the design with a sliding-window check over both the current and previous epoch, allowing cross-boundary duplicates to still be recognized and suppressed. *V3* adopts a dual-register Bloom-style design to strengthen collision behavior and reduce false positives, though its boundary behavior remains similar to V1 unless combined with additional temporal logic.

Implementation Choices. Several implementation choices keep the prototype simple while preserving the essential system behavior. FeBEx uses a two-hash approach, with one hash selecting the register index and the other acting as a verification key to reduce collision ambiguity. State expiry is implemented through epoch rotation rather than explicit deletion, which keeps the control plane lightweight. Tenant isolation is achieved through prefix-based LPM forwarding, avoiding per-device forwarding state. Finally, the switch performs clone-on-first-forward so that attribution is preserved even when redundant copies

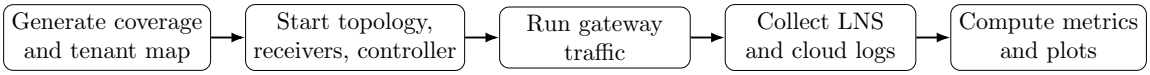

Figure 9: *End-to-end evaluation workflow.* Each experiment run generates the workload, executes the FeBEx topology, collects outputs from tenant and cloud endpoints, and computes the reported metrics.

are suppressed.

5 Evaluation

This section evaluates FeBEx using a Mininet-based BMv2 testbed and a Python P4Runtime controller. Our goal is to quantify how effectively FeBEx reduces redundant backhaul traffic while preserving correct delivery, tenant isolation, and attribution semantics. We additionally study the effect of key design parameters, including register size, epoch interval, and switch variant.

Evaluation Setup. We evaluate FeBEx on Mininet with BMv2 and a Python P4Runtime controller. The topology contains K gateway hosts generating packet copies according to a coverage matrix, one FeBEx BMv2 switch, M tenant LNS hosts, and one cloud host receiving cloned receipts. Traffic generation, packet capture, and metric computation are automated end-to-end. For each experiment point, we run both a *Dedup OFF* baseline, in which the switch performs only routing, and a *Dedup ON* mode, in which FeBEx actively suppresses duplicates.

Metrics. We report five primary metrics. *Backhaul savings* is computed as$1 - \frac{\text{packets}_{\text{ON}}}{\text{packets}_{\text{OFF}}}$, capturing the fraction of redundant traffic eliminated by FeBEx. *Delivery ratio* measures the fraction of unique uplinks that are correctly received at the intended tenant backend. *Tenant isolation* counts any cross-tenant forwarding violations. *Duplicate leakage* measures the number of duplicate packets that still pass through under deduplication. Finally, *throughput* is measured as the effective packet rate observed at the receiver.

Experiment Matrix. Our evaluation covers eight experiment families. E1 measures backhaul savings as a function of duplicate factor. E2 evaluates correctness through the delivery ratio of unique uplinks. E3 checks multi-tenant isolation. E4 studies scalability across increasing numbers of devices and gateways. E5 sweeps register size to understand state-provisioning tradeoffs. E6 sweeps epoch interval to evaluate temporal sensitivity. E7 verifies correctness of cloned payment receipts and gateway attribution. E8 compares the three switch variants V1, V2, and V3 under boundary and collision stress.

Configuration. Unless otherwise stated, experiments use a medium-city configuration with 100 devices, 10 gateways, and 4 tenants, together with fixed random seeds for reproducibility. Register size and epoch interval are swept only in E5 and E6, respectively. For E8, we keep the workload and topology fixed and vary only the switch variant so that differences can be attributed to the deduplication design itself.

E1: Backhaul Savings. Figure 10 shows the backhaul savings achieved by FeBEx as the duplicate factor increases. FeBEx provides strong bandwidth reduction, with savings rising from low-duplication scenarios to high-duplication scenarios, closely matching the expected trend. Savings increase from approximately 25% at low duplication to about 86% at high duplication, approaching the theoretical limit as more gateways hear the same uplink. Minor deviation at the highest stress point is consistent with BMv2 software effects under load rather than instability in the deduplication logic. Overall, these results show that in-network deduplication provides predictable and substantial backhaul reduction, with the largest gains in dense multi-gateway coverage regions.

E2: Correctness and Delivery Ratio. Figure 11 plots the delivery ratio of unique uplinks under deduplication. Delivery remains effectively perfect for normal operating points, and degradation appears only under extreme offered load where BMv2 software limitations dominate. In realistic and moderate-duplication settings, the delivery ratio remains approximately 1.0, indicating that FeBEx does not falsely suppress legitimate first copies. The small drop observed only at the most stressful operating point is consistent with software-switch bottlenecks rather than a flaw in the deduplication design. These results indicate that FeBEx preserves correctness across realistic operating ranges.

E3: Multi-Tenant Isolation. We next evaluate whether FeBEx preserves tenant boundaries while

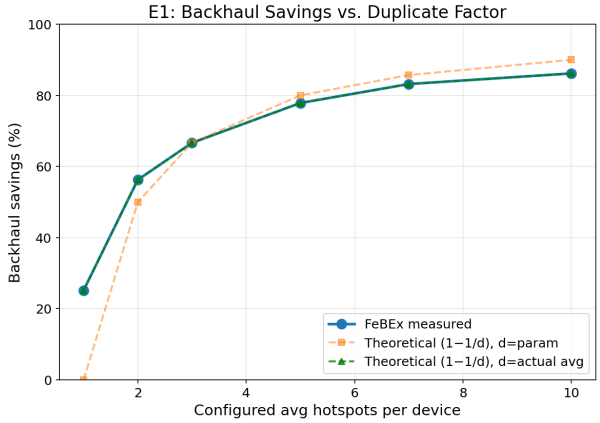

Figure 10: *Backhaul savings versus duplicate factor.* Savings increase with duplication and approach the theoretical limit as more redundant gateway copies are suppressed in-network.

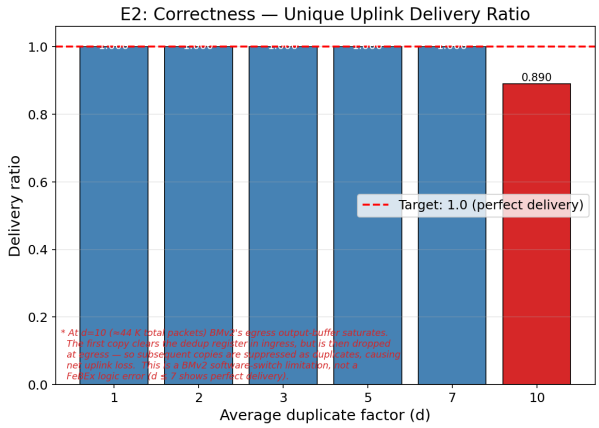

Figure 11: *Delivery ratio of unique uplinks under deduplication.* FeBEx maintains near-perfect correctness across normal operating points, with degradation only under extreme software-switch stress.

suppressing duplicates. Across the tested four-tenant scenario and thousands of forwarded packets, we observe no cross-tenant delivery violations. DevAddr-prefix-based LPM steering remains consistent under both routing-only and dedup-enabled modes. This confirms that duplicate suppression does not interfere with multi-tenant forwarding correctness, and that FeBEx can be deployed without breaking tenant isolation.

E4: City-Scale Scalability. Figure 12 evaluates FeBEx across increasing deployment sizes. Across small, medium, and large scenarios, FeBEx maintains substantial backhaul savings while throughput trends are dominated by software-switch capacity rather than algorithmic instability. Savings remain in a similar range, roughly from the high-50s to the high-60s percent, even as topology size increases from small to city-scale settings. Throughput rises initially and then drops at the largest emulation size, indicating saturation effects in BMv2 and Mininet. Thus, the deduplication behavior itself scales stably with topology growth, while the throughput ceiling is primarily an artifact of the software target.

E5: Register Sizing Sensitivity. Figure 13 studies the effect of register size on duplicate leakage and savings. As expected, larger register arrays sharply reduce collision-driven leakage. The largest improvement occurs when moving from very small tables to moderately sized ones, for example from 256 to 1024 entries, after which the benefit begins to plateau. Beyond 4096 entries, additional memory yields only diminishing returns for the evaluated workloads. Savings follow the same trend. These results support a practical rule of thumb for FeBEx deployment: provision register capacity at no less than 4× the expected active device count.

E6: Epoch Interval Sensitivity. Figure 14 evaluates the impact of epoch interval on deduplication effectiveness. Very short epochs increase boundary leakage because repeated packets are more likely to

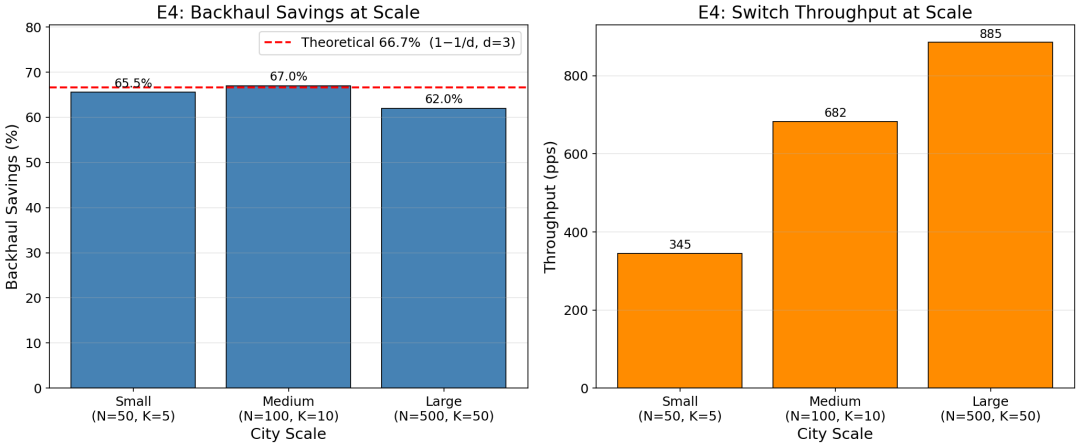

Figure 12: *Scalability across deployment sizes.* FeBEx maintains substantial backhaul savings as the number of devices and gateways increases, while throughput trends mainly reflect software-switch limits.

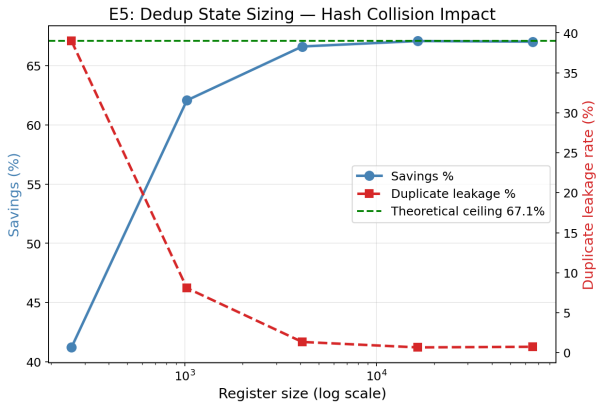

Figure 13: *Effect of register size on deduplication performance.* Larger register arrays sharply reduce duplicate leakage and improve backhaul savings, with diminishing returns beyond moderate table sizes.

straddle an epoch transition, while moderate epoch intervals provide stable suppression with minimal stale-state effects. In our experiments, leakage decreases monotonically as the epoch interval increases, with near-zero leakage observed at 10 seconds and above. Extremely aggressive intervals, such as sub-second or 1-second rotations, amplify boundary effects significantly. These results suggest that FeBEx should avoid overly short epochs; intervals of 5–10 seconds provide a robust operating region for the workloads considered here.

E7: Payment Receipt Attribution. We next verify that FeBEx preserves attribution semantics alongside duplicate suppression. Across the evaluated scenarios, cloned cloud receipts maintain a one-to-one correspondence with the first-forwarded tenant packets, and the gateway identifiers embedded in those cloned packets remain valid. This confirms that FeBEx does not merely suppress duplicates correctly, but also preserves the receipt semantics needed for Proof-of-Coverage and payment-related accounting.

E8: Variant Comparison. Figure 15 compares the three FeBEx switch variants under boundary-stress settings. V2, which uses a sliding-window check over the current and previous epoch, exhibits the best behavior when boundary leakage is the dominant source of error, since it suppresses duplicates that straddle an epoch transition. V3 improves collision-oriented behavior through its dual-register Bloom-style design, but it does not eliminate boundary leakage by itself, and therefore remains below V2 on this stress case. V1, as the baseline, is most exposed to cross-boundary duplicate leakage. These results suggest a clear deployment guideline: use V2 when boundary leakage is expected to dominate, and treat V3 as a complementary design for collision robustness rather than a direct solution to epoch-boundary effects.

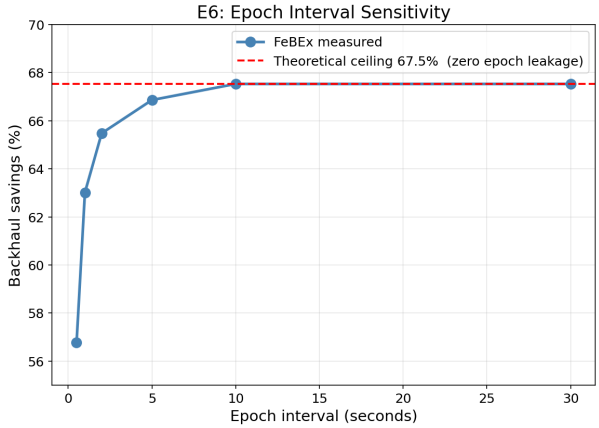

Figure 14: *Impact of epoch interval on deduplication effectiveness.* Moderate intervals reduce boundary leakage while maintaining simple, bounded-state expiry.

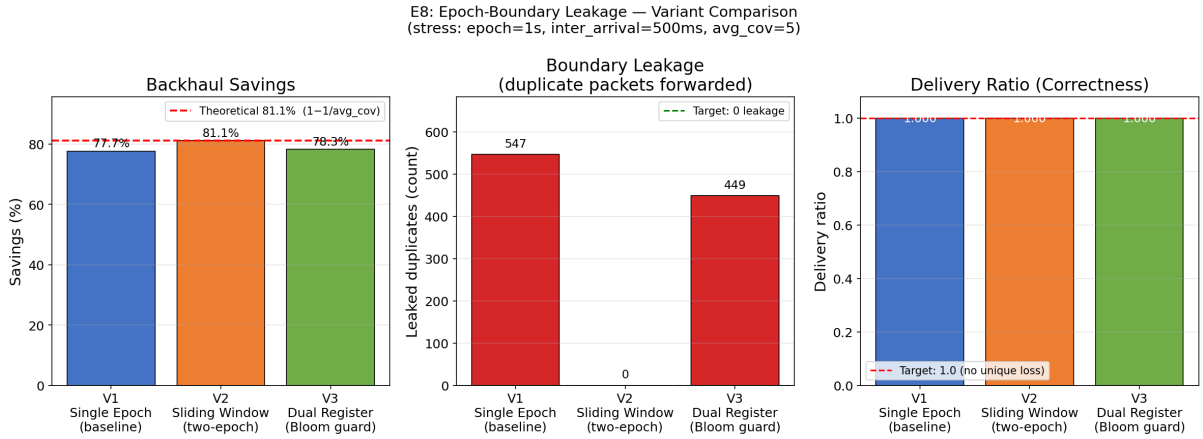

Figure 15: *Variant comparison under boundary stress.* V2 provides the strongest suppression of cross-boundary duplicates, while V3 primarily improves collision behavior rather than boundary robustness.

Threats to Validity. Our evaluation has three main limitations. First, BMv2 is a software target and can saturate under high offered load, so absolute throughput numbers should not be interpreted as hardware dataplane limits. Second, the packet format is intentionally simplified to isolate the backhaul deduplication problem rather than emulate a full LoRaWAN parsing pipeline. Third, our results emphasize the behavior of the deduplication logic and the resulting design trends, rather than the exact performance of a production hardware switch.

Evaluation Summary. Overall, the experiments show that FeBEx consistently reduces backhaul traffic, preserves correctness and tenant isolation, and maintains attribution semantics required for Helium-style deployments. The results also expose clear operational tuning knobs: register size controls collision-induced leakage, epoch interval governs temporal boundary effects, and the choice of switch variant determines whether the design prioritizes boundary robustness or collision resistance.

6 Discussion

Key Insights. Our evaluation highlights several practical lessons about in-network deduplication for federated LoRaWAN backhaul. First, compact register-based duplicate detection is sufficient to capture most of the available bandwidth savings. FeBEx does not require heavy-weight data structures or per-device state; a simple two-level hash design already achieves strong suppression performance under realistic operating conditions. Second, state provisioning is the dominant design knob. As shown in the register-sizing study, undersized tables lead to collision-driven leakage, while moderately provisioned tables recover most of the achievable savings. This makes register sizing a straightforward but important

deployment decision.

A third insight is that temporal policy matters almost as much as memory provisioning. Epoch-based expiry keeps the data-plane state bounded and simple, but overly aggressive epoch rotation introduces boundary leakage. Our results show that moderate intervals provide a good balance between freshness and suppression effectiveness. Finally, multi-tenant forwarding integrates naturally with deduplication. Because tenant steering is driven by `dev_addr` prefix matching rather than isolated per-tenant state, FeBEx can preserve routing separation without complicating the duplicate-suppression logic.

What FeBEx Contributes Beyond Backend Deduplication. Conventional LoRaWAN-style deduplication is typically performed at the backend after all packet copies have already traversed the network. FeBEx changes this placement. By suppressing duplicates at an aggregation switch, it reduces redundant backhaul traffic earlier in the path while still preserving correct delivery to tenant backends. In this sense, the main contribution of FeBEx is not a new deduplication primitive in isolation, but a change in where deduplication is performed and how it is integrated with forwarding and attribution.

Relative to more generic compact-filter approaches such as Bloom filters, the FeBEx design emphasizes simplicity and deterministic switch behavior. Rather than maintaining a more complex probabilistic structure, it uses an indexed verification-key design that maps well onto P4 register abstractions. This makes the implementation easier to reason about and aligns with the constraints of bounded switch memory. Relative to prior P4-based stateful services, FeBEx shows that programmable switches can do more than telemetry or load balancing: they can also support compact, time-aware filtering for federated IoT backhaul.

Limitations. FeBEx is evaluated on BMv2, which is a software switch rather than a hardware target. As a result, the reported throughput limits are shaped by CPU and Mininet overhead rather than by the intrinsic logic of the design. The trends observed in deduplication effectiveness, leakage, and state sensitivity are still meaningful, but the absolute performance numbers should not be interpreted as hardware dataplane bounds.

Our prototype also uses a compact FeBEx metadata header rather than parsing the full LoRaWAN packet stack. This abstraction is deliberate: it isolates the backhaul deduplication problem and keeps the evaluation focused on duplicate suppression, tenant steering, and attribution. However, a production implementation would need to integrate with the exact packet fields and parser constraints of a deployed LoRaWAN system. In addition, our current topology assumes a single aggregation point. That is appropriate for evaluating the core design, but real deployments may involve multiple regional aggregation points or more hierarchical routing structures. Finally, the current evaluation emphasizes duplicate suppression and backhaul efficiency under controlled workloads.

Practical Deployment Considerations. From a deployment perspective, FeBEx exposes three useful operational knobs. The first is register size: larger tables reduce collision-driven leakage, and our results suggest that sizing the table relative to the active device population is an effective rule of thumb. The second is the epoch interval: choosing it too aggressively increases boundary leakage, while moderate values provide stable suppression with bounded state lifetime. The third is the switch variant itself: if epoch-boundary effects dominate, the sliding-window design is preferable; if collision robustness is the main concern, the dual-register variant becomes more attractive.

7 Conclusion

Helium-style federated LoRaWAN deployments improve coverage and reliability by allowing multiple hotspots to receive the same uplink, but this design also causes redundant copies to consume backhaul bandwidth before deduplication is performed at the backend. This report presented FeBEx, a P4-programmable aggregation switch that moves duplicate suppression into the network itself. FeBEx combines compact register-based duplicate detection, epoch-driven expiry, prefix-based tenant steering, and attribution-preserving cloning in a single programmable pipeline.

Our evaluation shows that FeBEx substantially reduces redundant backhaul traffic while preserving correct delivery, tenant isolation, and receipt attribution semantics. It also identifies practical design guidance: register sizing strongly influences collision-driven leakage, moderate epoch intervals reduce boundary effects, and variant selection allows the system to trade off between temporal robustness and collision behavior. Although our prototype is evaluated on BMv2 and uses a simplified packet abstraction, the results demonstrate that in-network deduplication is both practical and effective.

Overall, FeBEx shows that duplicate suppression need not remain a backend-only software task. By moving it into the programmable aggregation layer, the network can reduce redundancy earlier, lower backhaul load, and preserve the operational semantics required by Helium-style deployments.

A Reproducibility and Commands

This appendix lists the primary build, run, and experiment commands used in this project.

A.1 VM Image

A prebuilt VM image for this project is available at:

https://nusu-my.sharepoint.com/:u:/g/personal/e1498124_u_nus_edu/IQCQhWoPrGzNQ6RUXvJJ9Re8AT6TT63cnfj5lIY_ROQTy24

A.2 Core Build and Run Commands

```
# Build default FeBEx switch (V1)
make build-febex

# Build boundary-leakage mitigation variants
make build-febex-v2
make build-febex-v3

# Run demo topology + controller
make run-febex

# Run automated switch tests
make run-tests-febex
```

A.3 Experiment and Evaluation Commands

```
# Run complete experiment suite
make run-experiments

# Run quick sweep
make run-experiments-quick

# Run one experiment only (example: E1)
make run-experiment-e1

# Evaluate logs and generate plots
make evaluate
```

A.4 Coverage and Visualization Commands

```
# Generate coverage matrix from YAML config
make generate-coverage \
  CONFIG=tasks/febex/configs/medium_city.yaml \
  OUTPUT=coverage.json

# Visualize coverage matrix
make visualize COVERAGE=coverage.json

# Generate Singapore interactive map visualization
make visualize-singapore
```

A.5 Cleanup Commands

```
# Stop and clean runtime artifacts
make run-clean

# Full cleanup (runtime + build)
make clean
```

References

[1] LoRa Alliance. Lorawan specification v1.0. LoRa Alliance Specification, 2015. https://lora-alliance.org/wp-content/uploads/2020/11/2015_-_lorawan_specification_1r0_611_1.pdf.

[2] LoRa Alliance. What is lorawan specification. LoRa Alliance Website, 2025. https://lora-alliance.org/about-lorawan-old/.

[3] LoRa Alliance. Lorawan backend interfaces v1.0. LoRa Alliance Specification, 2017. https://lora-alliance.org/wp-content/uploads/2020/11/lorawantm-backend-interfaces-v1.0.pdf.

[4] The Things Network. Lorawan architecture. Documentation, 2025. https://www.thethingsnetwork.org/docs/lorawan/architecture/.

[5] Martin Bor, Utz Roedig, Thiemo Voigt, and Juan M. Alonso. Do LoRa low-power wide-area networks scale? In *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, pages 59–67, 2016.

[6] Orestis Georgiou and Usman Raza. Low power wide area network analysis: Can LoRa scale? *IEEE Wireless Communications Letters*, 6(2):162–165, 2017.

[7] Helium. Helium documentation: Home. Helium Documentation, 2026. https://docs.helium.com/.

[8] Dhananjay Jagtap, Alex Yen, Huanlei Wu, Aaron Schulman, and Pat Pannuto. Federated infrastructure: Usage, patterns, and insights from "the people's network". In *Proceedings of the 21st ACM Internet Measurement Conference (IMC)*, pages 328–345, 2021.

[9] Helium. Helium – own the air. Official Website, 2026. https://www.helium.com/.

[10] Helium. Data-only hotspots. Helium Documentation, 2026. https://docs.helium.com/iot/data-only-hotspots.

[11] Helium. The mobile network – rewards. Helium Documentation, 2026. https://docs.helium.com/mobile/5g-on-helium.

[12] Weining Song, Stefanos Kaxiras, Thiemo Voigt, Yuan Yao, and Luca Mottola. Tada: Task decoupling architecture for the battery-less internet of things. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems*, SenSys '24, page 409–421, New York, NY, USA, 2024. Association for Computing Machinery.

[13] Louai Al-Awami. Network-coding-based forwarding for LoRaWAN gateways. *arXiv preprint arXiv:2205.03918*, 2022.

[14] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3):87–95, 2014.

[15] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Ananya Motoyama, Ashok Bigelow, Gibb Flajslik, Nick McKeown, and Guru Parulkar. In-band network telemetry via probe path selection. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*, 2016.

[16] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murali Sridharan. Conga: Distributed congestion-aware load balancing for datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, 2014.

[17] Xin Jin, Xiaozhou Li, Hal Zhang, Nate Foster, Jeongkeun Lee, Ion Stoica, and Dawn Song. Netcache: Balancing and caching for efficient content delivery. In *Proceedings of the 15th USENIX Symposium on Networked Systems Design and Implementation*, 2017.

[18] Ge Chen, Gaoxiong Zeng, and Li Chen. P4COM: In-network computation with programmable switches. *arXiv preprint arXiv:2107.13694*, 2021.

[19] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3rd edition, 2009.

[20] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.

[21] Jiaxi Ren and Rodrigo Fonseca. Silkroad: Making stateful layer-4 load balancing fast and cheap using switching asics. In *Proceedings of the 2021 ACM SIGCOMM Conference*, page 328–341, 2021.